

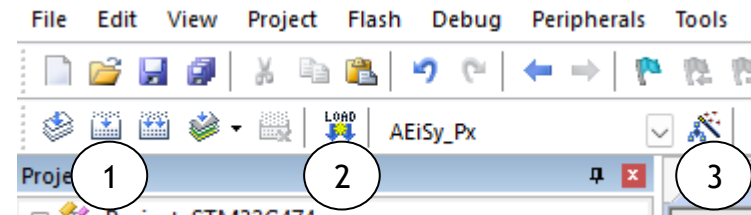
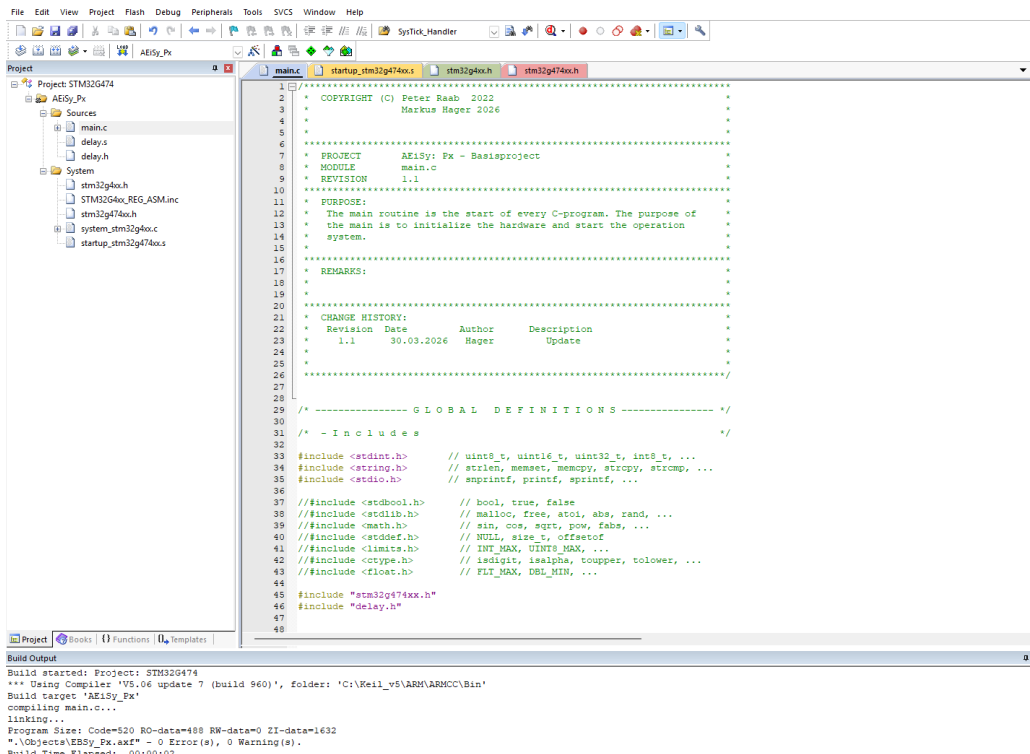
Autonome Eingebettete Systeme

Praktikum Einführung, LED und LCD Hardware

1. IDE Software & Basisprojekt
2. Hardware
3. LED Projekt
4. LCD Display

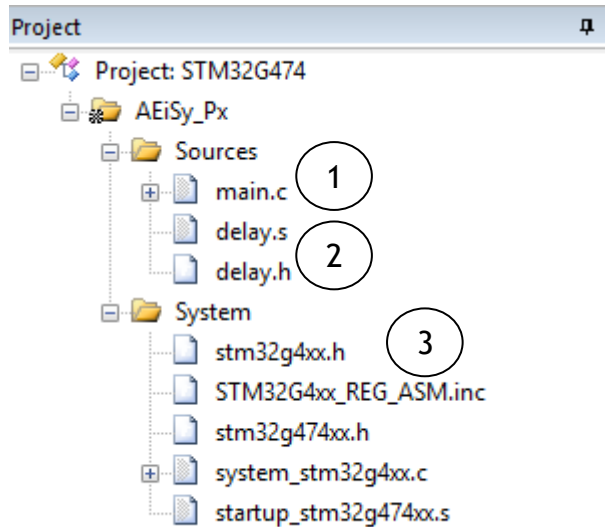
1. IDE Software

Keil μ Vision (gleich zur Vorlesung Mikrocomputertechnik)
Basisprojekt (s. Moodle) verwenden



- 1: Projekt übersetzen
- 2: Download zum Mikrocontroller
- 3: Projekteinstellungen

1. Basisprojekt



1 main.c: HauptEinstiegspunkt Ihrer Anwendung

2 delay: Definition und Implementierung einer Verzögerungsfunktion in Assembler

3

system_stm32g4xx.c
startup_stm32g474xx.s

SystemInit(): Clock-Setup, VTOR, FPU-Aktivierung -> **48 MHz**
Reset_Handler, Vektortabelle, Stack/Heap, Sprung zu main
Hinweis: 512 Byte heap, malloc() möglich
Register-Namen & Adressen zuordnen (GPIOA, UART, ...)
Register-Namen mit equ Definitionen für Assembler
inkludiert stm32g474xx.h, verwendet bei system_stm32g4xx.c

Hinweis: Im Projektordner sind bereits
C_Template.c und H_Template.h
für weitere Softwaremodule vorhanden.

Rückblick Assembler: Pin 0 an GPIOA konfigurieren und toggeln:

```
; 1. Clock für GPIOA aktivieren
ldr R0, =RCC_AHB2ENR
ldr R1, =1                ; Bit 0 = GPIOAEN
str R1, [R0]

; 2. PA0 als Ausgang konfigurieren
ldr R0, =GPIOA_MODER
ldr R1, [R0]              ; MODER lesen
ldr R2, =0xFFFFF0FC      ; Bits 1:0 maskieren (PA0)
and R1, R1, R2
ldr R2, =0x00000001       ; 01 = Output
orr R1, R2
str R1, [R0]

; 3. Endlosschleife - PA0 toggeln
loop
    ldr R0, =GPIOA_ODR
    ldr R1, [R0]
    eor R1, R1, #1        ; Bit 0 toggeln
    str R1, [R0]

    ; Verzögerung
    ldr R2, =500000
delay
    subs R2, R2, #1        ; verkürzt mit Statusflag-Update
    bne delay

    b loop
```

Nun in C: Pin 0 an GPIOA konfigurieren und toggeln

```
// 1. Clock für GPIOA aktivieren
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;

// 2. PA0 als Ausgang konfigurieren
GPIOA->MODER &= ~GPIO_MODER_MODE0_Msk; // Bits 1:0 löschen
GPIOA->MODER |= (1<<0) // GPIO_MODER_MODE0_0 01 = Output

// 3. Endlosschleife - PA0 toggeln
while (1)
{
    GPIOA->ODR ^= GPIO_ODR_OD0; // Pin0 toggeln

    // Verzögerung
    for (volatile uint32_t i = 0; i < 500000; i++);
}
```

Beispiel: stm32g474xx.h

```
5852 | #define GPIO_MODER_MODE0_Msk (0x3UL << GPIO_MODER_MODE0_Pos) /*!< 0x00000003 */
```

Hinweis: HAL als weitere Option nach LCD Display

Quelle: <https://github.com/STMicroelectronics/stm32g4xx-hal-driver>

```
#include "stm32g4xx_hal.h"

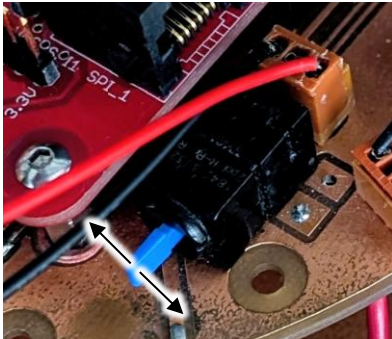
// 1. Clock aktivieren
__HAL_RCC_GPIOA_CLK_ENABLE();

// 2. PE11 als Ausgang konfigurieren
GPIO_InitTypeDef GPIO_InitStruct = {0};
GPIO_InitStruct.Pin = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; // Push-Pull
GPIO_InitStruct.Pull = GPIO_NOPULL;
GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

// 3. Pin toggeln
HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_0);
```

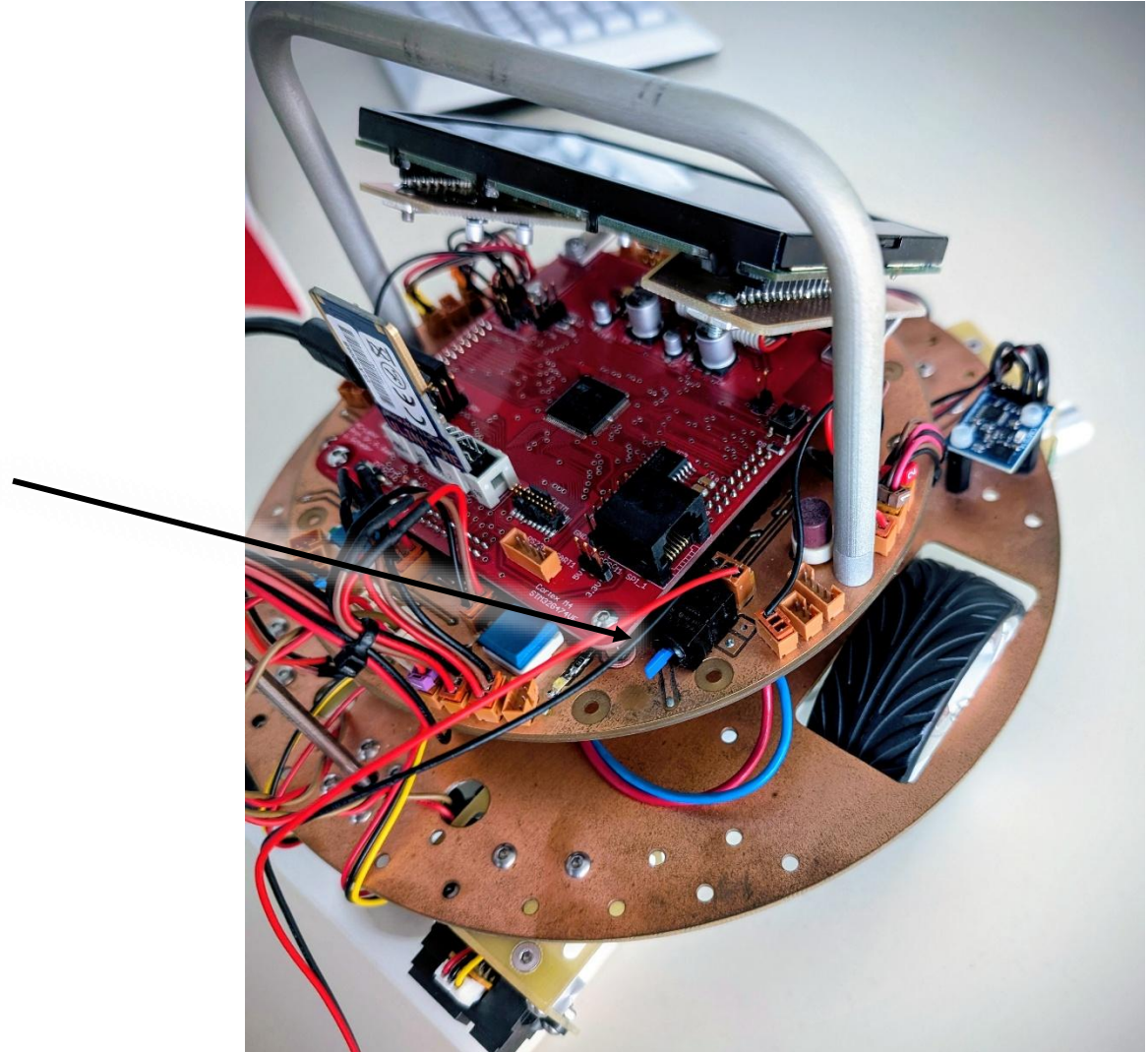
2. Hardware

Power Schalter



3 Stellungen:

- ↖ Netzversorgung
- Mittelstellung: AUS
- ↘ Batterieversorgung

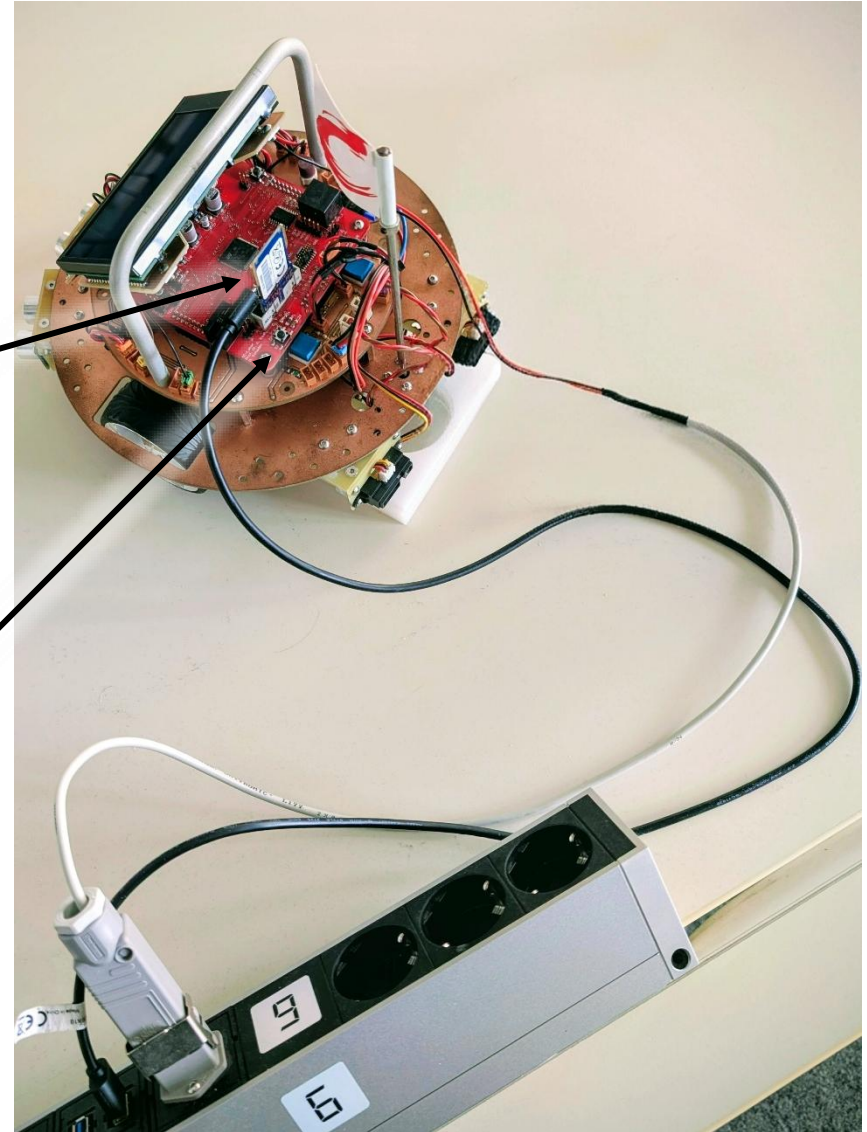


2. Hardware

Programmierer
Debugger

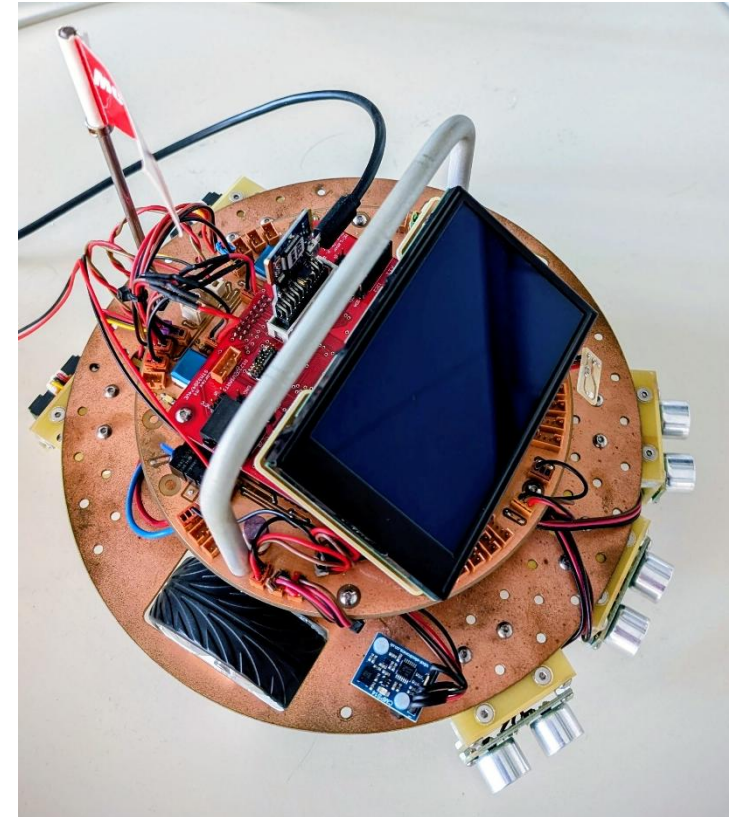


Reset-Button



2. Hardware tech. Daten

- STM32G474VET6 Cortex M4 (CPU)
 - LD1117S33TR 3,3V Regulator
 - Toshiba T6963C LCD controller Display 240x128
 - L298 (dual full-bridge driver)
 - RW-1205S 5V DC/DC Converter
-
- | | |
|-------------------------------|---------------|
| • Raddurchmesser | 66 mm |
| • Radabstand | 175 mm |
| • max. Drehzahl Getriebemotor | 60 U/min |
| • Last Drehzahl Getriebemotor | 52 U/min |
| • max. Drehmoment | 180 Ncm |
| • Inc. Radencoder | 120 Inc. |
| • Versorgungsspannung | 12V (Motor) |
| • DC/DC-Wandler | 12V/5V |



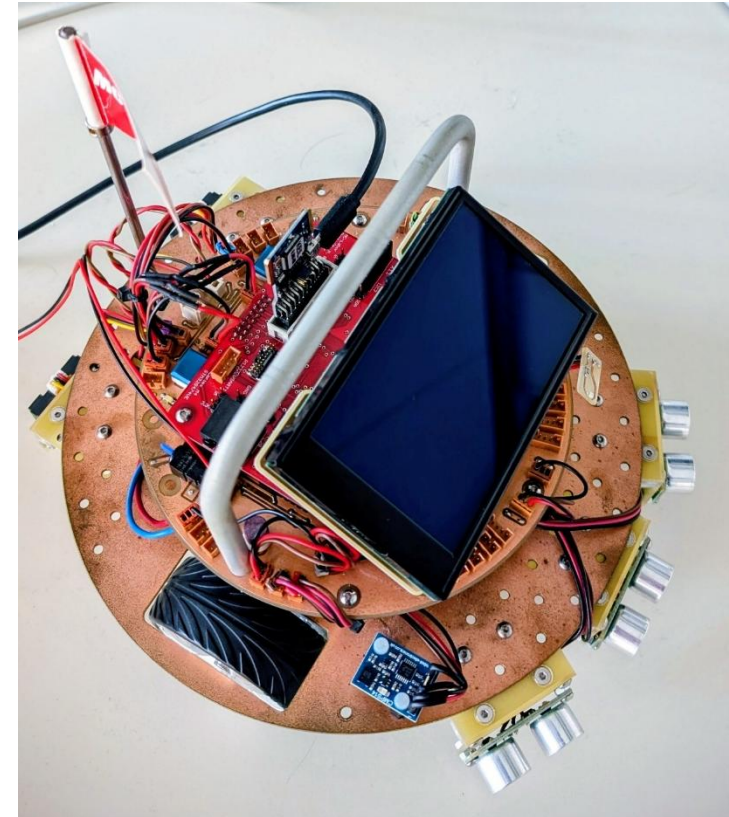
2. Hardware tech. Daten

Sensoren

- Ultraschallsensor SRF05 - Ultra-Sonic Ranger
- IR-Sensor GP2D120
- Radencoder GP1A30R
- Taster
- Kompassmodul CMPS12
- RC5 IR-Empfänger TSOP 31236

Aktoren

- Motor RB35 12V 1:100
- LED OSRAM-LS_M67K ROT
- Display EA DIP 240-7
(240x128 Pixel mit Controller T6963C)

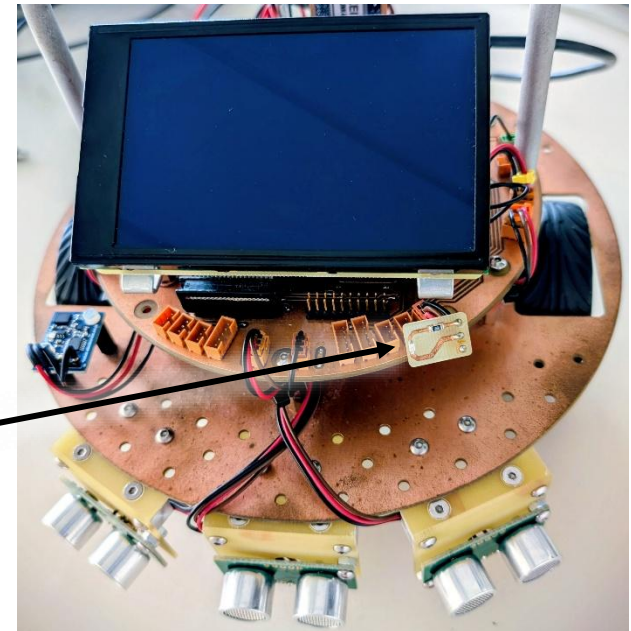
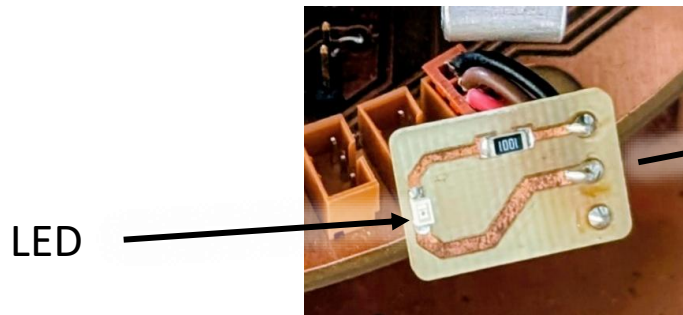


Schaltpläne etc. s. Moodle

3. LED Projekt: Blinklicht

Aufgaben LED a:

1. Öffnen Sie das Basisporjekt
2. Bestimmen Sie aus dem Schaltplan den GPIO Pin, mit dem die LED verbunden ist.
3. Initialisieren Sie exklusiv diesen GPIO Pin als Ausgang.
4. Toggeln Sie die LED (an - aus) in einem Intervall von 1 s (= 1 Hz). Implementieren Sie die Verzögerung mit einer Schleife oder nutzen Sie die delay Funktion



3. LED Projekt: Blinklicht mit Softwarearchitektur

Aufgaben LED b:

1. Für eine nicht blockierende Zeitbasis benötigen Sie einen Timer.
Empfehlung: nutzen Sie hierfür Timer6 oder den SysTick-Timer
2. Konfigurieren Sie den Timer so, dass er bspw. alle 100ms einen Interrupt auslöst und toggeln Sie die LED über diese Funktion.
3. Schauen Sie sich die zum Basisprojekt zugehörige Event.c/h an.
Vollziehen Sie den Quellcode eigenständig nach und verstehen Sie ihn!
4. Passen Sie Ihr Softwareprojekt nun wie folgt an:
 - Definieren Sie eine aussagekräftige Event-ID und lassen Sie den Timer diesen Event generieren.
 - Kapseln Sie die LED-Logik in ein Softwaremodul. Implementieren Sie eine passende Schnittstelle, bspw. eine einmalige Konfigurations-Funktion und eine Funktion, welche einen Event entgegennimmt.
(die Konfiguration können Sie auch über einen Event implementieren s. Folie L3-14 im Foliensatz „Software Architektur“)
 - Passen Sie die main.c entsprechend der Event-Logik an.
Orientieren Sie sich an Folie L3-14 im Foliensatz „Software Architektur“



Upload in Moodle!

4. LCD Display

Lernziele:

- **Inbetriebnahme eines LC Displays**
→ Darstellung von Betriebszuständen und Sensorwerten des MC-mobile
- **Inbetriebnahme des Flexible Static Memory Controllers**
(FSMC@STM32G474)
- **Entwicklung einer Graphikbibliothek**
(Ausgabe von Strings und Bitmaps, Zeichnen von Linien, Kreisen, ...)
- **Integration** in bestehende **Embedded Software**

4. LCD Display

EA DIP240B-7KLW

240 x 128 Pixel

LCD Controller: T6963c

8 Bit Datenbus

Text und Graphik

128 Zeichen (Generator)



4. LCD Display

Graphikspeicher:

Beispiel:

Pixel in 4. Zeile / 5. Spalte

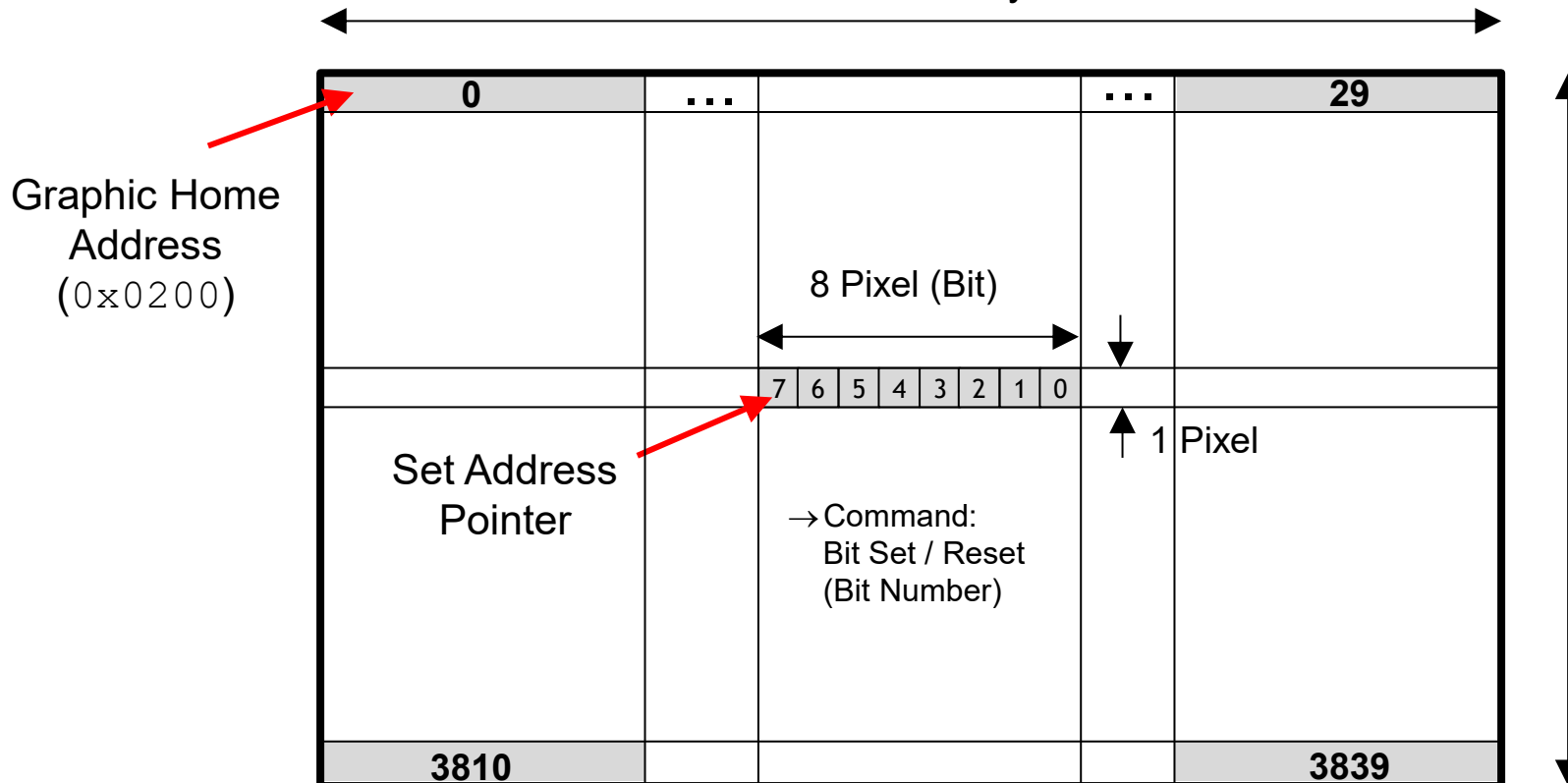
Adresse = (Zeile-1) · 30

+ (Spalte-1) / 8

= 90 + 0 = 90

Bitnummer = 7 - (Spalte-1) mod 8

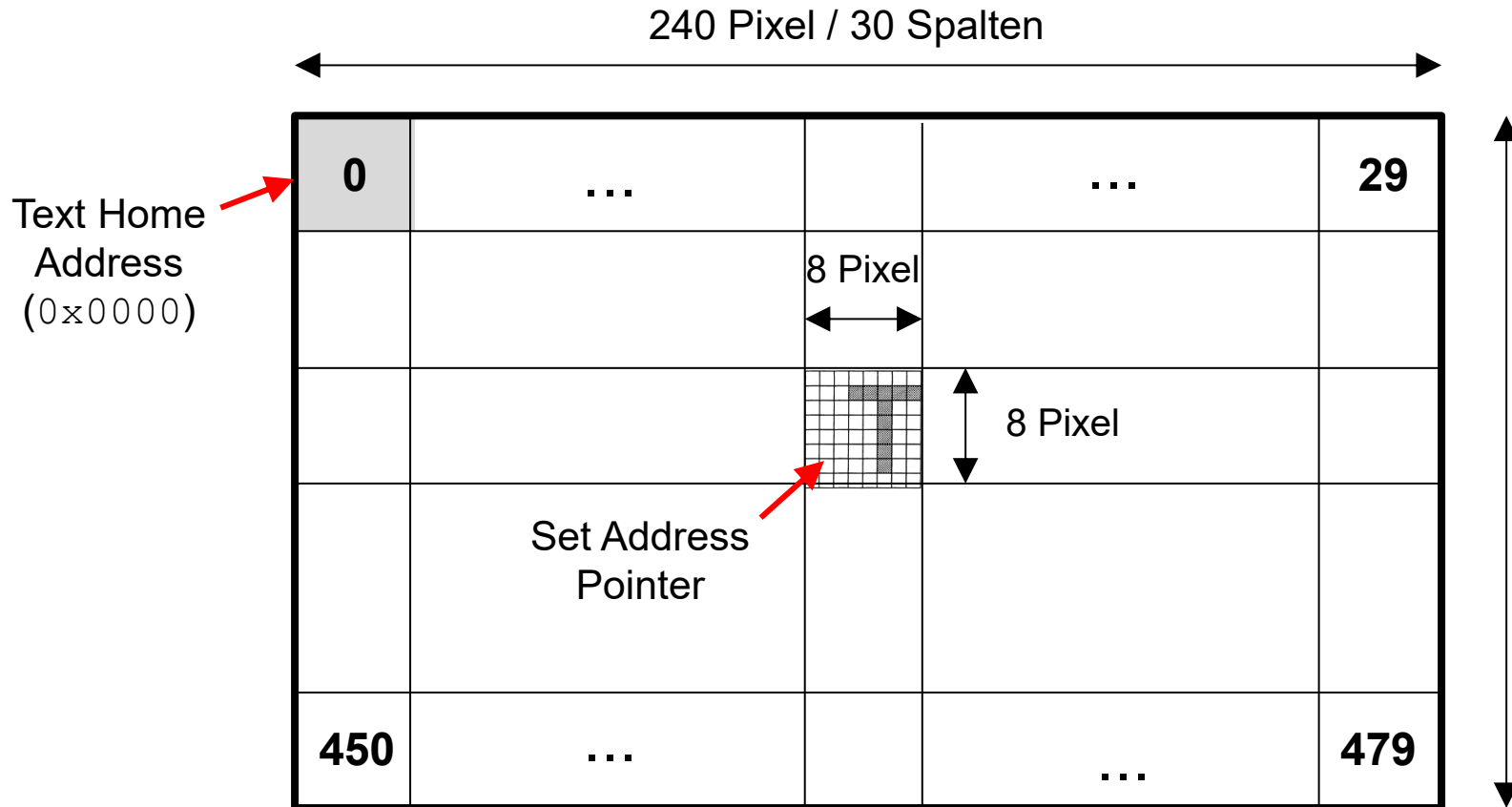
240 Pixel / 30 Byte



4. LCD Display

Textspeicher:

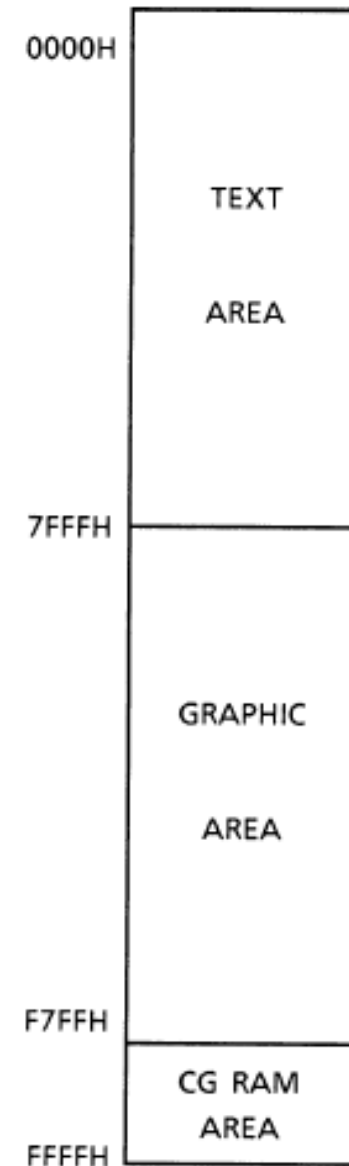
Beispiel:
Zeichen in 4. Zeile / 5. Spalte
Adresse = (Zeile-1) · 30
+ (Spalte-1)
= 90 + 4 = 94



4. LCD Display

Adressbereich:

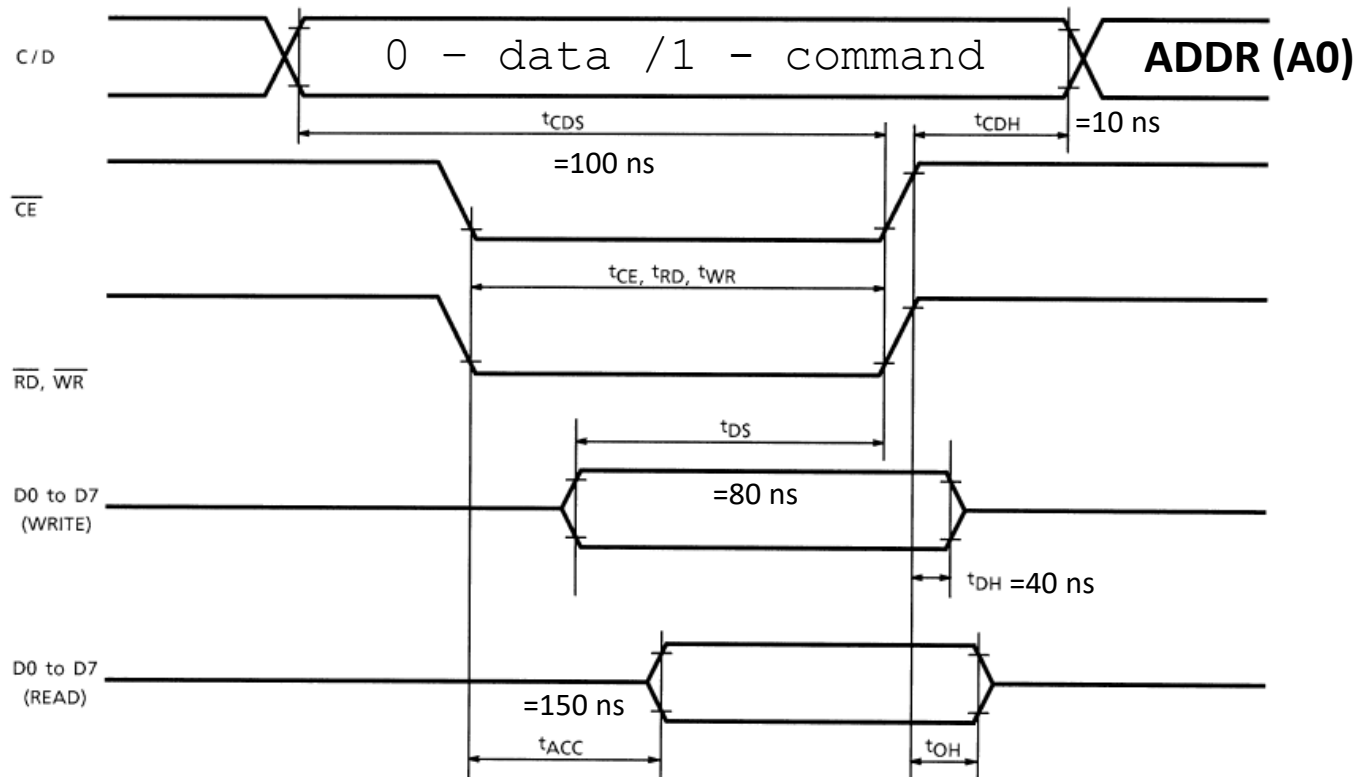
- Speichern von Anzeigedaten (Text, Graphik)
- drei Bereiche:
 - **Text Area:** Ausgabe von Texten
 - **Graphic Area:** Ausgabe von Graphiken
 - **CG (character generator) Area:**
Definition von Zeichensätzen
- können frei zugewiesen werden (max. 64 kB)



4. LCD Display

0x6000:0000 - data
0x6000:0001 - command

Grafik-Controller: T6963c - Busschnittstelle und Timing



Pin Name	I / O	Functions
C / D	Input	$\overline{WR} = L$ C / D = H: Command Write C / D = L: Data Write
		$\overline{RD} = L$ C / D = H: Status Read C / D = L: Data Read

4. LCD Display

Kommunikation:

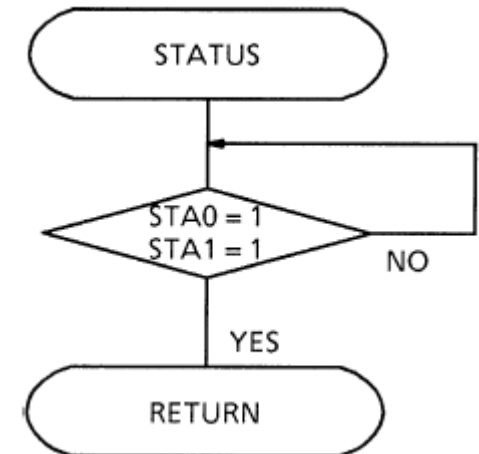
(1) Status Read

$\overline{RD}$	L
$\overline{WR}$	H
$\overline{CE}$	L
C / D	H
D0 to D7	Status word

Read address:
0x6000:0001

MSB				LSB			
STA7 D7	STA6 D6	STA5 D5	STA4 D4	STA3 D3	STA2 D2	STA1 D1	STA0 D0

STA0	Check command execution capability	0: Disable 1: Enable
STA1	Check data read / write capability	0: Disable 1: Enable
STA2	Check Auto mode data read capability	0: Disable 1: Enable
STA3	Check Auto mode data write capability	0: Disable 1: Enable
STA4	Not used	
STA5	Check controller operation capability	0: Disable 1: Enable
STA6	Error flag. Used for Screen Peek and Screen copy commands.	0: No error 1: Error
STA7	Check the blink condition	0: Display off 1: Normal display



4. LCD Display

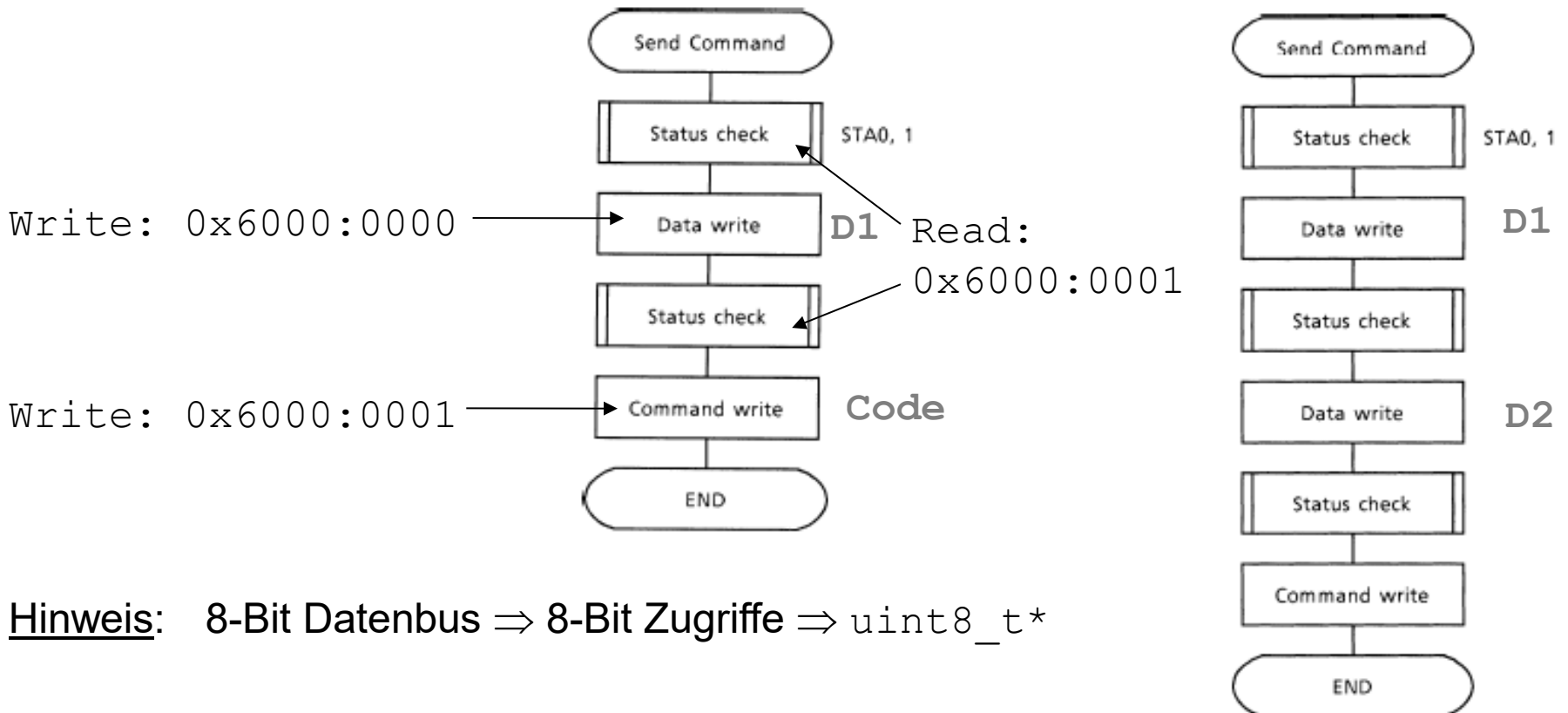
Kommunikation:

Procedure for sending a command

(2) Send Command

a) The case of 1 data

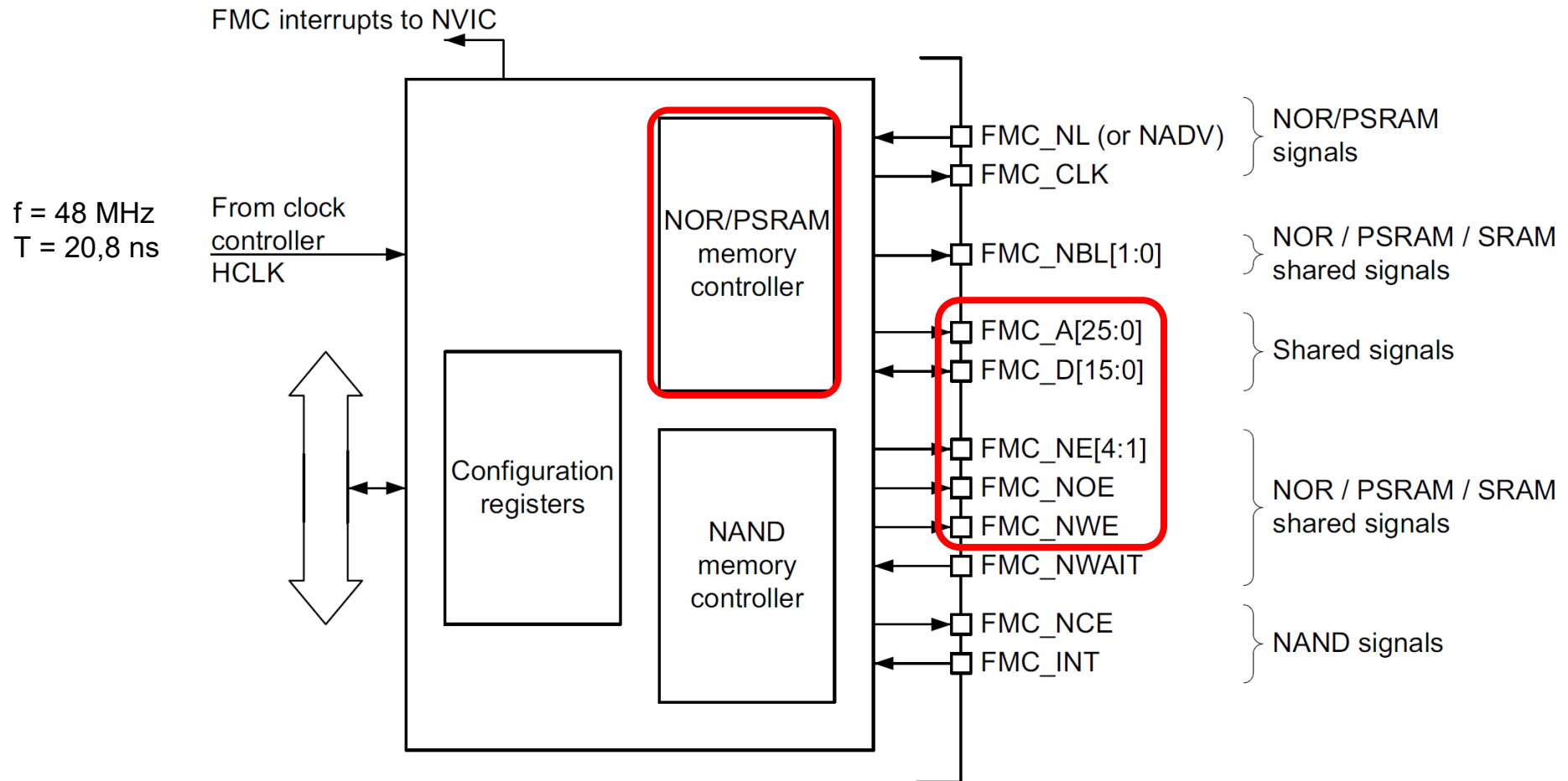
b) The case of 2 data



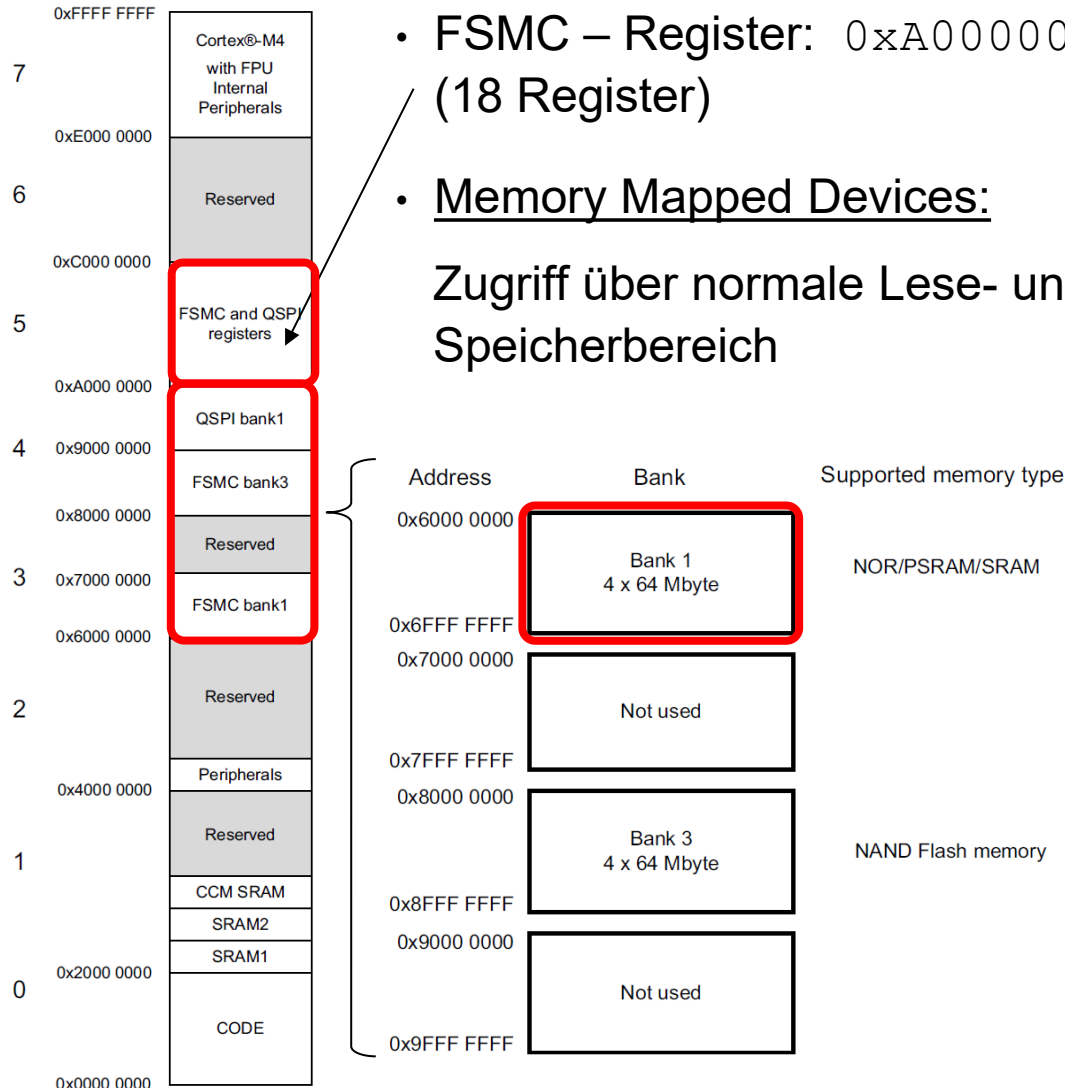
Hinweis: 8-Bit Datenbus $\Rightarrow$ 8-Bit Zugriffe $\Rightarrow$ `uint8_t*`

4. LCD Display: FSMC

Block Diagramm:



4. LCD Display: FSMC

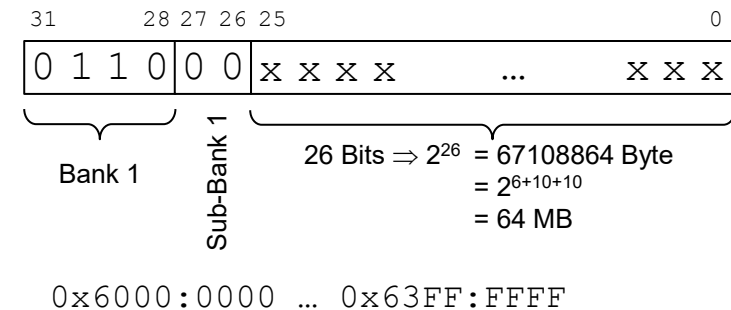


- FSMC – Register: 0xA0000000 – 0xA0000FFF (18 Register)

- Memory Mapped Devices:

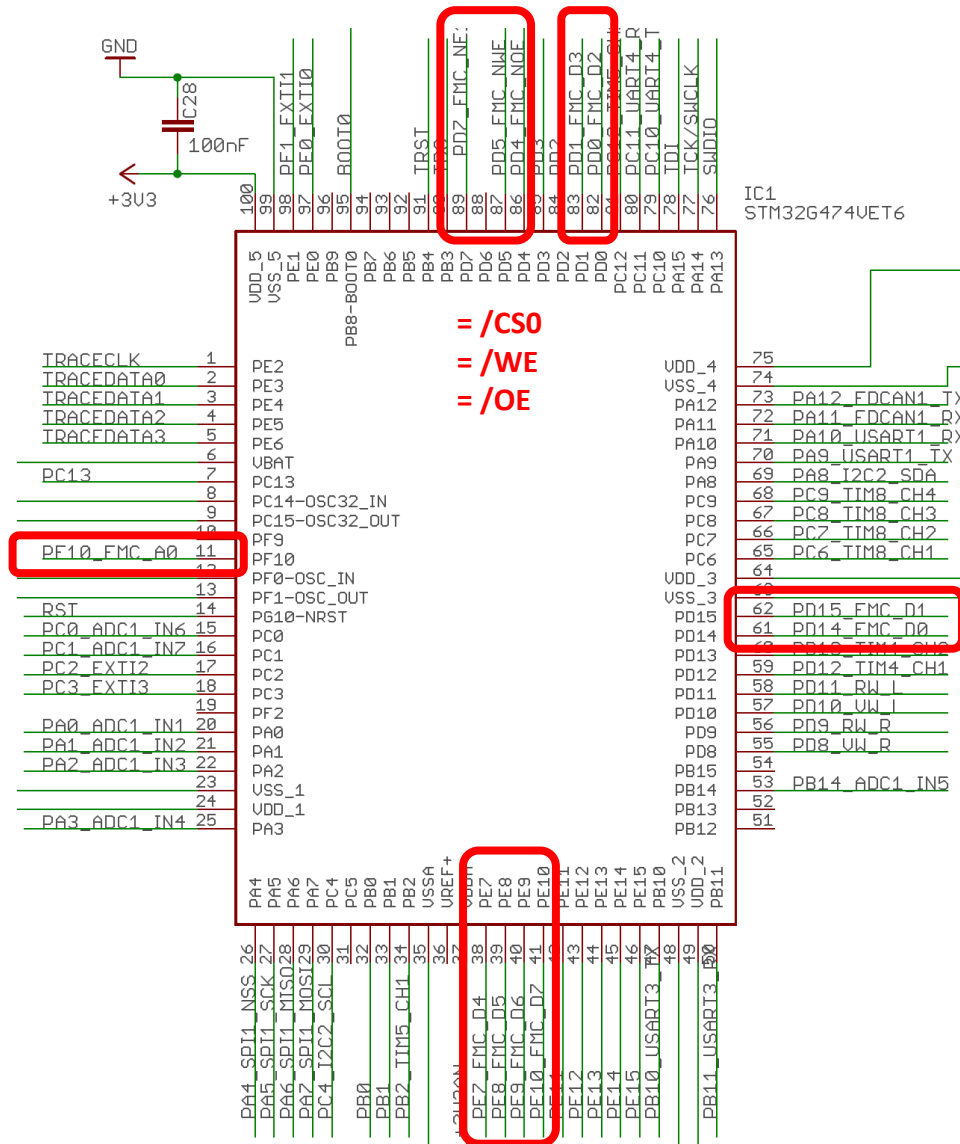
Zugriff über normale Lese- und Schreibzugriffe im gegebenen Speicherbereich

Adressaufbau:



STM32G474:

C/D =



4. LCD Display: FSMC

• Konfiguration:

→ SRAM/NOR-Flash chip-select control register for bank 1 – BCR1

(FMC_Bank1_R->BTCR[0])

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NBLSET[1:0]		WFDIS	CCLK EN	CBURST RW	CPSIZE[2:0]		
								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ASYNC WAIT	EXT MOD	WAIT EN	WREN	WAIT CFG	Res.	WAIT POL	BURST EN	Res.	FACC EN	MWID[1:0]		MTYP[1:0]		MUX EN	MBK EN
rw	rw	rw	rw	rw		rw	rw		rw	rw	rw	rw	rw	rw	rw

→ SRAM/NOR-Flash chip-select timing register for bank 1 – BTR1

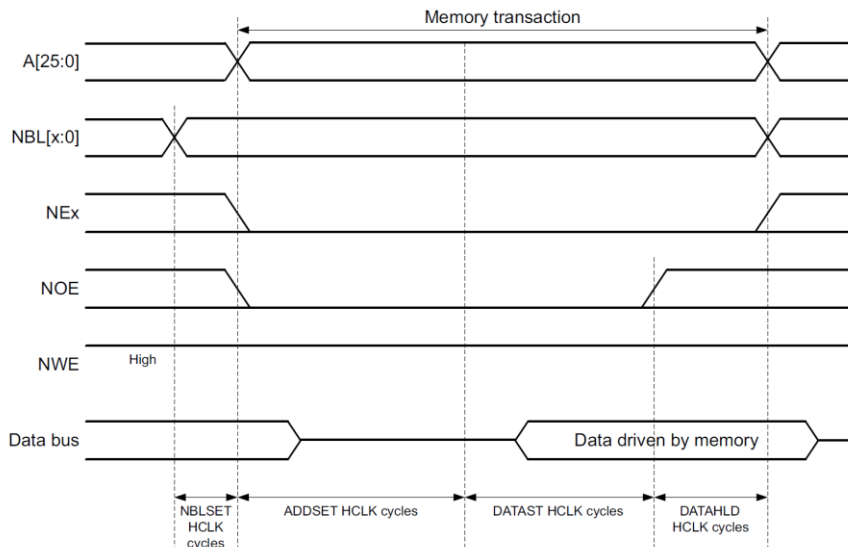
(FMC_Bank1_R->BTCR[1])

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
DATAHLD[1:0]		ACCMOD[1:0]		DATLAT[3:0]				CLKDIV[3:0]				BUSTURN[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATAST[7:0]								ADDHLD[3:0]				ADDSET[3:0]			
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

4. LCD Display: FSMC

• Timing:

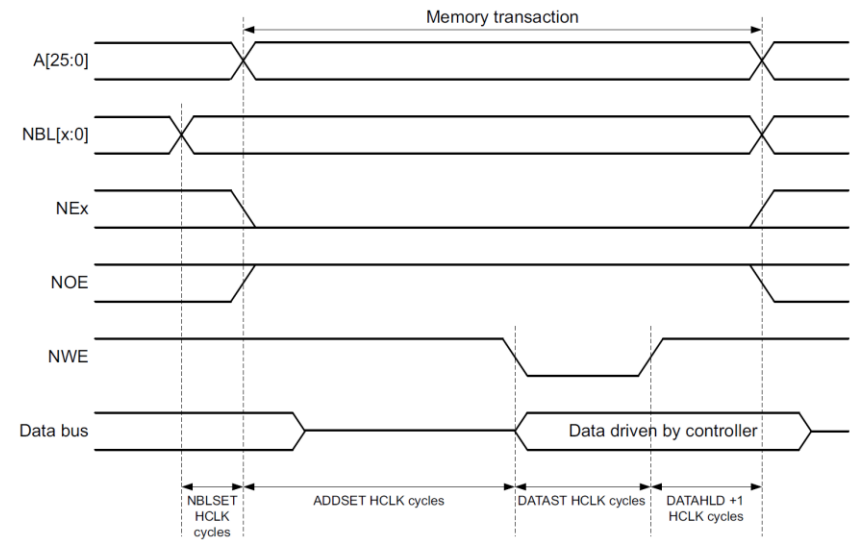
→ Read:



min. 150 ns
≥ 8 HCLK Takte

min. 10 ns
≥ 1 HCLK Takt

→ Write:



min. 80 ns
≥ 4 HCLK Takte

Hinweis: Defaultwerte der Timings sind ausreichend groß!

ABER: DATAST = 255 / ADDSET = 15 / DATAHLD = 0 !!!

4. LCD Display: FSMC

- Reference Manual RM0440, Kapitel 19
- Konfiguration der notwendigen Pins:
 - Clock der Pins aktivieren
 - RCC Register
 - und konfigurieren
 - GPIOx MODER/AFR (→ AF12)
- Konfiguration des FSMC:
 - Clock enable (RCC→AHB3ENR)
 - FMC_Bank1_R→BTCR[0]
 - Memory bank enable (Bit 0) = 1
 - Memory type (Bits 3:2) = 00
 - Memory data bus width (Bits 5:4) = 00
 - Write enable (Bit 12) = 1
 - Write Fifo Disable (Bit 21) = 1
 - FMC_Bank1_R→BTCR[1]
 - DATAHLD (31:30) = 3
 - DATAS (15:8) = 255
 - ADDSET (3:0) = 15

Pins	Funktion
PF10	Address bus A0
PD14	Data Bus D0
PD15	Data Bus D1
PD0	Data Bus D2
PD1	Data Bus D3
PE7	Data Bus D4
PE8	Data Bus D5
PE9	Data Bus D6
PE10	Data Bus D7
PD4	Output enable /OE
PD5	Write enable /WE
PD7	Chip select /CS0

Hinweis:
verbleibende Bits der Register bleiben 0!

4. LDC Display

Inbetriebnahme: *LCD Controller*

- T6963C – Datenblatt (→ Moodle)
- Initialisierung:
 - Set Text Home Address → 0x0000
 - Set Text Area to 30₁₀: ein Zeichen ist 8 Pixel breit
⇒ 30 Zeichen pro Zeile
 - Set Graphic Home Address → 0x0200
 - Set Graphic Area to 30₁₀: ⇒ 30 Bytes pro Zeile
 - Activate EXOR Mode: überlappende Pixel des Text- und
Graphikspeichers werden XOR verknüpft
 - Set Display Mode for Text and Graphics → ON

4. LCD Display

Inbetriebnahme: C-Code

Lesen, bspw. für Statusüberprüfung

```
uint8_t status = *((volatile uint8_t*)0x60000001);
```

oder mit

```
#define LCD_DATA_ADDR      ((volatile uint8_t*)0x60000000)
```

```
#define LCD_CMD_ADDR       ((volatile uint8_t*)0x60000001)
```

Schreiben:

```
*((volatile uint8_t*)0x60000001) = cmd;
```

```
*((volatile uint8_t*)0x60000000) = data;
```

4. LDC Display: Aufgabe

Entwicklung einer Graphik-Bibliothek

- Ausgabe eines Strings auf dem Display:
→ Kommandos: *Set Address Pointer / Data Write and Increment ADP*
- Ausgabe eines Bitmaps im Graphik-Speicher:
→ C-Code: `unsigned char picture[3840] = {...};`
→ Konvertierung einer BMP-Datei in Bytewerte im Quelltext.
Programm **Image2Code.exe** (→ Moodle)
- weitere Graphik-Funktionen:
 - **Pixel** setzen/löschen
 - **Rechtecke** zeichnen (Rahmen, ausgefüllt)
optional:
 - Kreise/Ellipsen zeichnen
 - ...



Upload in Moodle!